

🔍 Méthode : Comprendre un message d'erreur (Traceback)

Lorsqu'une exception survient, Python interrompt l'exécution et affiche une trace d'appels (*traceback*) qui se lit *de bas en haut* :

- ❶ Lire la *dernière ligne en premier* : elle donne le *le type de l'erreur* et un message explicatif précis (ex : `IndexError: list index out of range`).
- ❷ Consulter la *ligne incriminée* .
 ⚠ Dans le cas particulier d'une erreur de syntaxe `SyntaxError`, l'erreur réelle peut se situer à la ligne *précédente* (par exemple si une parenthèse n'a pas été refermée)

🛠 1. Erreurs de syntaxe (SyntaxError)

Une erreur de syntaxe empêche l'exécution du script avant même son démarrage, ces erreurs sont parfois repérées par l'éditeur de texte.

- *Oubli des deux-points* : à la fin d'un en-tête d'instruction conditionnelle, de boucle ou de fonction :

❌ Code erroné :

```
1 if x > 0
2     print("positif")
3 def carre(n)
4     return n * n
```

✅ Code corrigé :

```
1 if x > 0:
2     print("positif")
3 def carre(n):
4     return n * n
```

- *Confusion entre affectation = et comparaison ==* :

❌ Code erroné :

```
1 if a = 3:
2     print("égal à 3")
```

✅ Code corrigé :

```
1 if a == 3:
2     print("égal à 3")
```

- *Parenthèse, crochet ou guillemet non fermé* :

❌ Code erroné :

```
1 val = (2 * x + 1%)
2 print(val) # SyntaxError ici !
```

✅ Code corrigé :

```
1 val = (2 * x + 1)
2 print(val)
```

- *Noms de variables invalides* : un nom de variable ne peut ni commencer par un chiffre (ex : `1ere_valeur`), ni contenir de tiret (ex : `total-points` confondu avec une soustraction), ni être un mot réservé Python (`for`, `in`, `def`, `class`, `return`, `pass`, `while`, `import`, ...).

🛠 2. Erreurs d'indentation (IndentationError)

En Python, l'indentation structure les blocs de code. Toute incohérence bloque l'exécution.

- ❶ Toujours utiliser la touche `⇧ + ⇞` pour ajouter des indentations (et la combinaison `⇧ + ⇞` pour en retirer). Na *jamais* tenter d'aligner les blocs en utilisant la barre d'espace.

🛠 3. Erreurs de type (TypeError)

Survient lors de l'application d'un opérateur ou d'une fonction à un objet d'un type non approprié. Cette erreur survient notamment dans les cas suivants (liste non exhaustive)

- *opération interdite entre types incompatibles :*

❌ Code erroné :

```
1 s = "Total : " + 42
2 # Concaténation str + int
3 L = [1, 2, 3] + 2
4 # Addition d'une liste et d'un entier
```

✅ Code corrigé :

```
1 s = "Total : " + str(42)
2 # Conversion explicite
3 L.append(2)
4 # Méthode appropriée
```

- Flottant utilisé comme indice ou paramètre de `range` :

❌ Code erroné :

```
1 milieu = len(L) / 2
2 val = L[milieu] # float !
```

✅ Code corrigé :

```
1 milieu = len(L) // 2
2 val = L[milieu] # int !
```

- *variable de type None*. L'utilisation d'une *méthode* sur un objet de Python (comme un *append*) ne renvoie PAS l'objet mais `None`, de même une fonction qui ne renvoie pas de résultat, renvoie `None`.

❌ Code erroné :

```
1 L = [3, 1, 2]
2 L = L.append(4)
3 # L vaut maintenant None !
4 print(len(L)) # TypeError !
```

✅ Code corrigé :

```
1 L = [3, 1, 2]
2 L.append(4)
3 # append modifie L en place
4 print(len(L)) # Affiche 4
```

⚠ Même piège avec `L.sort()` (modifie en place et renvoie `None`) à distinguer de `sorted(L)` (renvoie une nouvelle liste triée).

- `object is not callable` : on utilise la syntaxe d'appel à une fonction `<nom_fonction>(arguments)` alors que `<nom_fonction>` n'est pas une fonction. L'exemple typique est d'utiliser `ma_liste(5)` au lieu de `ma_liste[5]`
- `takes X positional arguments but Y were given` : mauvais nombre d'arguments passés lors de l'appel d'une fonction.
- `'tuple' / 'str' object does not support item assignment` : tentative de modification d'un élément d'un type *immuable* (ex : `texte[0] = 'A'` ou `mon_tuple[1] = 10`).

✎ 4. Erreurs d'indice (IndexError)

L'exception **IndexError**: `list index out of range` survient lorsqu'on tente d'accéder à un élément avec un indice $i \notin \llbracket -\text{len}(L); \text{len}(L) - 1 \rrbracket$. Voici quelques exemples :

- Boucle allant un cran trop loin :

✖ Code erroné :

```
1 # Indice len(L) inexistant !
2 for i in range(len(L) + 1):
3     print(L[i])
```

✔ Code corrigé :

```
1 # Indices de 0 à len(L)-1
2 for i in range(len(L)):
3     print(L[i])
```

- Accès au successeur $L[i+1]$: la borne supérieure de la boucle doit être réduite d'une unité :

✖ Code erroné :

```
1 # Pour i = len(L)-1, i+1 déborde !
2 for i in range(len(L)):
3     if L[i] == L[i+1]:
4         print("doublon")
```

✔ Code corrigé :

```
1 # Arrêt à len(L)-2
2 for i in range(len(L) - 1):
3     if L[i] == L[i+1]:
4         print("doublon")
```

- Affectation directe sur une liste vide / non dimensionnée :

✖ Code erroné :

```
1 carres = []
2 for i in range(10):
3     carres[i] = i**2 # IndexError !
```

✔ Code corrigé :

```
1 carres = []
2 for i in range(10):
3     carres.append(i**2)
```

- ❗ Pour pré-allouer une liste de taille n , écrire `carres = [0] * 10`, puis `carres[i] = i**2`.

Les tranches `L[debut:fin]` ne lèvent *jamais* d'**IndexError**, même si `fin` dépasse la longueur de la liste (l'extraction s'arrête simplement au dernier élément).

✎ 5. Erreurs de nom et portée (NameError, UnboundLocalError)

Survient lorsqu'on utilise un identificateur non défini ou non accessible dans la portée courante.

- **NameError**: `name '...' is not defined` :

- Faute de frappe ou casse non respectée : Python est sensible à la casse (`total`, `Total` et `TOTAL` sont 3 variables distinctes).
- Utilisation avant affectation : variable manipulée avant sa ligne d'initialisation.
- Oubli d'un `import` : utiliser `sqrt(x)` sans `from math import sqrt` préalable.
- Variable locale inaccessible hors de sa fonction :

✖ Code erroné :

```
1 def moyenne(a, b):
2     res = (a + b) / 2
3     return res
4
5 moyenne(12, 16)
6 print(res) # NameError !
```

✔ Code corrigé :

```
1 def moyenne(a, b):
2     res = (a + b) / 2
3     return res
4
5 m = moyenne(12, 16)
6 print(m) # Récupérer le résultat
```

- Variable définie dans un bloc conditionnel non exécuté : si une variable n'est affectée que dans un `if`, elle n'existera pas si la condition est fausse (lui donner une valeur par défaut avant le test).

- **ModuleNotFoundError**: `No module named '...'` : nom de bibliothèque mal orthographié (ex : `import maths` au lieu de `math`).

6. Valeurs, arithmétique et assertions (ValueError, ZeroDivisionError)

L'argument est du bon type, mais sa valeur est invalide ou non acceptée pour l'opération demandée.

- **ZeroDivisionError**: division by zero : division `/`, quotient entier `//` ou modulo `%` avec un dénominateur nul (cas d'une liste vide lors du calcul d'une moyenne, ou d'un pivot nul).
- **ValueError**: invalid literal for int() : tentative de conversion d'une chaîne ne représentant pas un entier :

❌ Code erroné :

```
1 n = int("3.14") # ValueError !
2 m = int("dix")  # ValueError !
```

✅ Code corrigé :

```
1 n = int(float("3.14")) # Vaut 3
2 m = 10
```

- **ValueError**: math domain error : argument hors du domaine de validité mathématique d'une fonction (ex : `math.sqrt(-4)`, `math.log(0)`, `math.asin(2)`).
- **ValueError**: not enough / too many values to unpack : déballage d'un tuple ou d'une liste avec un nombre incompatible de variables à gauche de l'affectation :

❌ Code erroné :

```
1 x, y = (1, 2, 3) # Trop de valeurs
2 a, b, c = [10, 20] # Pas assez
```

✅ Code corrigé :

```
1 x, y, z = (1, 2, 3)
2 a, b = [10, 20]
```

- **AssertionError** : levé lorsqu'une instruction `assert <condition>` échoue (la précondition testée n'est pas vérifiée lors de l'appel).

7. Fichiers, dictionnaires et attributs (KeyError, AttributeError)

- **FileNotFoundError**: [Errno 2] No such file or directory : tentative d'ouverture avec `open("fichier.txt")` d'un fichier inexistant, mal nommé (ex : faute dans l'extension) ou situé dans un autre répertoire (problème de chemin relatif).
- **KeyError**: 'cle' : accès direct à une clé qui n'existe pas dans le dictionnaire :

❌ Code erroné :

```
1 d = {"a": 1, "b": 2}
2 val = d["c"] # KeyError !
```

✅ Code corrigé :

```
1 d = {"a": 1, "b": 2, "c": 3}
2 val = d["c"] # "c" est bien une clé !
```

- **AttributeError**: '...' object has no attribute '...' : Utilisation d'une méthode inexistante sur le type manipulé (ex : `L.append(5)` au lieu de `L.append(5)`).

8. Récursion et boucles infinies (RecursionError et terminaison)

- **RecursionError**: maximum recursion depth exceeded : trop grand nombre d'appels récursifs imbriqués (limite par défaut à 1000) :
 - Oubli du cas de base (condition d'arrêt) : la fonction s'appelle indéfiniment.
 - Pas de récurrence incorrect : l'argument ne converge pas vers le cas de base (ex : appel `f(n)` au lieu de `f(n - 1)`).

❌ Code erroné :

```
1 def fact(n):
2     # Oubli du cas de base !
3     return n * fact(n - 1)
```

✅ Code corrigé :

```
1 def fact(n):
2     if n == 0:
3         return 1
4     return n * fact(n - 1)
```

- **Boucle while infinie** : le programme s'exécute sans fin et bloque sans message d'erreur (utiliser alors la combinaison de touche `Ctrl+C` pour interrompre le programme) :
 - Oubli de la mise à jour de la variable de contrôle dans le corps du `while` (ex : omission de `i += 1`).
 - Condition d'arrêt stricte sur un flottant : `while x != 1.0` : avec un pas `x += 0.1` peut ne jamais s'arrêter en raison des erreurs d'arrondi de l'arithmétique flottante.